

Continuous, Lean, and Wasteless: Minimizing Lead Time from Development Done to Production Use

Timo Lehtonen¹, Terhi Kilamo², Sampo Suonsyrjä², and Tommi Mikkonen²

¹Solita Plc. FI-33000 Tampere, Finland

²Tampere University of Technology, FI-33720 Tampere, Finland

timo.lehtonen@solita.fi, terhi.kilamo@tut.fi, sampo.suonsyrja@tut.fi, tommi.mikkonen@tut.fi

Abstract—Modern software organizations invest substantial effort in building and automating their tool chain. The goal is to maximize both the speed of development, and how rapidly new software is deployed. This paper presents results from a descriptive and exploratory single case study from an ongoing project of Finnish company Solita. Based on data from version control and production logs, we investigate the feature flow in the project to study the effect of lean processes and the continuous deployment tool chain to waste produced by the deployment pipeline. We find that flow efficiency can be improved simply by minimizing the idle time the feature waits in the production process after its implementation has been finalized. This reduction of waste benefits both end users and developers – the users get access to new features, and the developers receive timely feedback.

I. INTRODUCTION

Software companies are paying increasing attention to their prompt ability to deliver value to end users. The goal of Lean Software Development (LSD) [1] – or applying the principles of Lean manufacturing to software – is to postpone decisions to the latest possible moment while delivering value as soon as there is value to deliver. Already completed features are constantly improved based on data regarding the actual use of software. In the spirit of LSD, actions that do not serve these goals and delays in delivering the features to end users can be regarded as waste. Extensive infrastructure is needed to maximize the speed of development and deployment. Such pipeline, including a version control system, build and test servers, and automated production installations, enables delivering features to end users more rapidly than ever before [2].

In this paper we study what kind of LSD processes and tools direct the development towards rapid continuous deployment, and how giving timely information throughout development can support team workflows. As concrete data, we use the project "Lupapiste", freely translated "Permission desk", a web site for managing municipal authorizations and permissions created by Solita, a mid-sized Finnish software company specializing in the design and implementation of web systems, analytics, and business intelligence. We investigate the feature flow of the project in regard to the lean concept of waste. To recognize waste produced in the deployment pipeline, we have applied the Value Stream Mapping (VSM) [3] method to the project's continuous delivery pipeline.

II. BACKGROUND

Lean Software Development (LSD) refers to applying the principles of lean manufacturing in the development of software systems [1]. In LSD, independent teams deliver software where new features are constantly made available to end users, and features that already exist are constantly improved based on data regarding the actual use of software. Actions that do not serve these goals, as well as delays deploying the features, can be regarded as waste, which is to be eliminated.

One way to advocate LSD is Continuous Delivery, or the practice of keeping the software in such a condition that it can be deployed to staging and production environments at any time [4]. This enables faster feedback, increased productivity, and improved communication. Continuous Deployment (CD) [5] takes one step further from delivery by automatically deploying software as it gets done and tested. When taking CD to the extreme, new features are deployed to the end users several times a day [6].

Regardless of actual deployment, continuous software development requires a tool chain – a deployment pipeline, which uses an automated set of tools from code to delivery. The role of these tools is to make sure each stakeholder gets a timely access to the things they need. In addition, the pipeline provides a feedback loop to each of the stakeholders from all stages of the delivery process. In this paper we use the concept of a deployment pipeline based on the ideas of Humble and Farley [4] as the reference model. The pipeline they propose is widely used by practitioners especially in the field of web application development both in the industry and in open source projects. The approach of [4] is to make use of a distributed version control system (DVCS), build once, and then deploy the same binary to any execution environment.

III. WASTE AND THE DEPLOYMENT PIPELINE

In manufacturing context, value stream is a collection of actions that are required to bring a product through the two main process flows, starting with a raw material and ending with the customer [3]. *Production* is the flow from raw material to the customer, and *design* is the flow from concept to launch. In the context of the deployment pipeline, the production flow is essential. It is the process where the specifications are converted into the elements of the software that produce business value to the stakeholders, as flow items go through the deployment pipeline. The design flow is out of the scope

for this paper, as it occurs outside the deployment pipeline, mainly in requirements management and design phases.

In a lean process, resources add value to a *flow unit* [7]. A flow unit is typically material, information or people. *Lead time* is the time it takes for the flow unit to flow through the process from beginning to end. In software development context, a flow unit is a new feature implemented into the software, which may be, for example, new functionality, bug fix, or technical refactoring. The resources that add value to the feature are the team of developers, customer-side stakeholders, and the users of the software. The servers of the deployment pipeline also add value to the feature as they can verify the code automatically in the different environments. This shortens the lead time as no human work is needed for verifying that the system is still working after the updates.

The efficiency of a lean process is divided to two different views: *flow efficiency* and *resource efficiency* [7]. *Flow efficiency* describes how much a *flow unit* is processed in a specific time frame – here, the time it takes for a feature from completion of development to its first activation in production use. *Resource efficiency* is about maximizing the usage of the resources, which in general means maximizing the time that resources, in this case people or machines, spend executing their work. As this issue is hardly a key question in software development, we focus on flow efficiency of new features from development to production use.

In LSD, any activity in the value stream that does not produce value to the flow unit can be regarded as waste [7]. Thus, waste within software development ranges from unnecessary code getting written to any delays that occur during the development process [1]. In this paper, the focus is on waste produced by delays in the deployment pipeline.

All activity that happens in a value stream [3] can be divided into two categories: value added (VA) activity and non-value added (NVA) activity. We define value added activity related to a new feature in the context of this paper very strictly. From our viewpoint, the only value added activity is the development phase. Once development is completed, the time until the first use in production environment is considered waste. In this context, we have identified three sources of waste – extra steps in quality assurance, waiting for use after deployment, defects in production – each contributing to extended lead time. This is wasteful as with short lead times, the feedback cycle is also short and effective, whereas with longer lead times, there are more flow units flowing simultaneously, which forces the developers to context switching and restarting unfinished tasks.

Extra Steps – Testing and quality assurance: Depending on the type of application being developed, extensive testing can be interpreted as extra processing steps on an already finished feature, i.e. waste. If new features are implemented, the application is verified to work with extensive regression testing. The risk that the application does not work at all, or causes harm that should then later be cleaned up, is usually rather low. Instead, when the team deploys the software into the production environment, hidden bugs and defects rise to prominence. Moreover, the team gets timely feedback from the

actual users of the system. The developers may not understand the user needs thoroughly and the deployment makes the conflicting interpretations visible.

Countermeasures: Whenever new features are done, deploy them to the production the next working day. Plan the deployment schedule with the customer to, for example, once or twice a week. The definition of done can even be pushed to mean delivered [4]. In many cases, developer testing locally is enough, and the users in the production environment can do the rest of the testing, often without considering (or even knowing) this. If there are bugs, the users of the system will report them. Furthermore, the monitoring platform may catch some of the defects and report them automatically.

Waiting, including the time that a feature waits for first use after deployment to production environment: By definition, inventory is waste, and in this case, every execution environment is an inventory where a feature may be pending. If a new feature has been deployed into an execution environment and its testing process is slow, waste may be produced. Moreover, if a new feature has been deployed into production environment and is waiting for use, the time spent waiting is in this context regarded as waste. The reasons why the users do not use the new feature can vary. For example, it is possible that the users do not know about the addition of the new feature, or they may not need the new feature yet.

Countermeasures: Plan the customer acceptance testing in advance, so that it happens with a short feedback cycle after the development is done. Communicate effectively with the testers and inform them about the new features. When the testers know what to test and how to test it the possible acceptance testing performed by the testers should proceed fluently. Demand the customer to test rapidly in the quality assurance environment and inform the users about the new features effectively. Help the users find the new features by keeping them up to date efficiently. One way to inform the users in some settings is to contact them personally. Others include sending email announcements or including an introductory wizard to the user interface of the system to guide users through the newly added features.

Defects in production environment not caught by developer tests: If regression and acceptance testing of a new feature have not been extensive enough, defects that arise in the production environment produce waste. Part of the waste is the context switch – if the developer has already started the implementation of another feature, the context switch to fixing a critical bug in production environment, and then switching back to the earlier task is waste. Another form of waste is the extra communication needed for example for browsing the issue management system, reading the bug reports and asking for more information from bug reporters.

Countermeasures: One way to avoid context switching is to agree on a duty officer where one team member at a time is responsible for handling the urgent bug requests. This way, developers' context switching – obviously waste – is avoided.

The fundamental problem behind all the three wastes of deployment pipeline — Extra Processing Steps, Waiting, and

Defects — is the long lead time. The first two of the three wastes lengthen the lead time of a new feature being implemented directly. If the feature is waiting in an inventory, or if it is processed further in vain, the lead time is prolonged. When the feature is processed in the testing phase and when the feature is waiting for deployments or waiting for users, the lead time is prolonged. The third waste, defects in production, also prolongs the lead time, because a broken new feature was incomplete, and the lead time of the new feature can be interpreted to end only when the feature was completely implemented. Moreover, the management work needed for handling the defects is waste, which prolongs the lead time.

The larger the amount of flow units flowing simultaneously, the more choices there are for context switching and for restarting unfinished tasks. Context switch for a programmer is an expensive operation [8] that should be avoided in order to be more effective in the development process. Restarting a previously unfinished task produces waste in terms of doing the same things again.

Finally, the latency between the deployment and the first use in production makes the feedback cycle even longer. With latency we mean the extra time a flow unit spends in a non-value added activity. For example, if a new feature is waiting for the deployment to the production environment for 10 extra days, the total lead time is prolonged in vain from the flow units point of view. There are many reasons for deployment latency. For example, the current version may have been refactored in a way that breaks the regression tests.

IV. CASE LUPAPISTE

A descriptive and exploratory single case study industrial case study [9] was conducted to investigate waste in a state-of-the-art pipeline within a two month's time frame of actual development work. Focus was on what kind of a lean development process and toolset direct the development processes towards CD and how describing timely workflow information during development can help to detect bottlenecks. The application "Lupapiste" is available at <https://www.lupapiste.fi/>. The permission types applied contain e.g. building permissions, environmental permissions and digging permissions. There is also a lightweight free form information request for the citizen to ask geolocated questions from the municipal authority. The municipal authority can start a dialog with the citizen through the chat functionality of the application, and then, if needed, the citizen can apply for a formal building permit with detailed information on the location and properties of the building including architectural drawings and plans as attachments. Also some other parties, like an architectural designer, can be invited to the application easily by email. When the applicant thinks that everything is ready to be submitted, the information is then transferred to a municipal decision making system. Some time later, when the authorities have handled the application usually in a formal meeting, the system sends email to the citizen, who is then redirected back to the application to see the details of the decision. The usage of the system nation-wide is emerging and some tens of

municipalities have taken the system into use. The amount of municipal authority users is in the hundreds and the amount of citizens using Lupapiste is hundreds per month.

The project was started in 2012. The supplier of the system is Solita Plc., a mid-sized Finnish software company. The end user of the system consists of various stakeholders, with various interests. The Environmental Ministry of Finland owns the project code and acts as a customer in some new functionalities needed to the system. The business model is unusual as the marketing of the system has been delegated to the supplier, providing an opportunity for the supplier to collect transactional costs of the permissions applied by the citizens and companies. This motivates the supplier on marketing the application to the municipal authorities in Finland. Indeed, in comparison to the old fashioned paper based system, Lupapiste has drastically shortened the lead times of building applications. For example, as reported at the web site <https://www.lupapiste.fi/>, in the city of Järvenpää the lead time of the permission handling process was shortened from three weeks to three days.

At the time of research (Fall 2014), the project team consisted of four developers, a user experience (UX) designer and a project manager co-located in a single workspace at the supplier. On the management level there are four more persons in different roles. Some team members have an ownership of certain parts of the system, but the knowledge is actively transferred inside the team by rotating these continuously and for example by applying agile practices like pair reviewing of code to spread out the knowledge in a continuous manner. The team makes use of a custom Agile process that includes features from Scrum, but with Lean heritage where also Kanban tools, e.g. WIP-limit or kanban cards [10] are applied. The case project's pipeline environment consists of a personal local development environment, the common development environment, a testing environment, a quality assurance environment, and the production environment. The deployments to the different environments are managed through the version control system, which then provides the accurate and automatic metadata to measure the process.

The study is based on quantitative data and descriptions of the development processes and the pipeline collected from the developer team. Additional informal discussions were carried out to gain knowledge on team practices for deployment. Empirical data of the case project was collected from the project's DVCS – Mercurial VCS (<http://mercurial.selenic.com/>) – and monitoring system Splunk (<http://www.splunk.com>). The data gathered was then exported and analyzed using statistical tool R (<http://www.r-project.org>).

To depict the project's development and deployment pipeline direct, informal discussions with the developers were held by one of the researchers. The researcher works in the case company and has been working in the case project earlier, so the current team could be contacted directly by the researcher to collect the required information. Software measurement as a process of representing software entities, like processes, in quantitative numbers [11], was further applied

in the study. We also applied descriptive statistics to get an understanding of the data collected.

Researcher bias in this types of study settings is unavoidable. The researcher responsible for the description of the case projects deployment pipeline has been working in the case project. Another internal threat is selection bias: the case project brings with it a plethora of aspects, such as development and customer culture, that affect the study. A clear threat to external validity of the research is that it investigates a single case. Thus, it poses a threat to the generalization of the results.

V. RESULTS AND DISCUSSION

Identifying Waste: We applied Value Stream Mapping (VSM) to recognize the waste in the lean process used together with the deployment pipeline. The VSM current state map [3] is depicted in Figure 1, with an extension depicting a branching model [12] on a conceptual level. The figure is a combination of a process description combined to the inventories of the deployment pipeline. The terms used in the diagram are derived from the branching model and from Kanban [13] terminology and the terms that software engineering practitioners use. The Kanban terminology comes from the team’s development practices. The team has for example used a Kanban board to visualize the state of the process [13] in order to recognize the bottlenecks on the deployment pipeline.

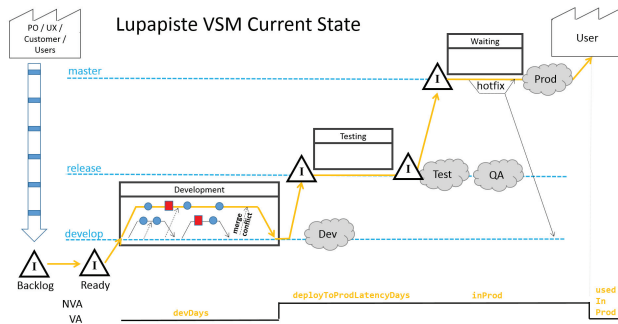


Figure 1. A value stream mapping of the case project.

The development of a new feature is started when developer starts working on the feature from the ready buffer. Physically the ready buffer is a column on a Kanban board indicating the items there are ready to be implemented. The product owner fills the ready buffer, selecting the most important features at the moment from the product’s backlog. The backlog in turn is filled with user needs and requirements from the customer, or with requirements up with which a UX designer or the product owner have come. The ready buffer always contains a small number of the most important features needed to the system being developed which the developer can start working on. In the case, the concrete day-to-day software development process of the team is based on the use of the version control system and the Hudson continuous integration system (<http://hudson-ci.org/>).

At the beginning of the development of a new feature, its ticket on the Kanban board is moved to the “development” column, marking the start of the programming phase. The developer opens a new local feature branch in the distributed version control system. The state of the development branch has to be green according to the information radiator [14], implying all tests in the development branch must pass. If broken code has been committed to the development branch, a lot of waste is generated, and, in addition, other team members are not able to start the development of new features or synchronizing with the development branch. Each developer works on a single item at a time by writing the source code with comprehensive tests. Value is added in a continuous manner with local commits [15].

The development process includes programming activities that introduce value. If the lead time of development phase is long, the risk of merge conflicts rises, because the longer the development lead time of a feature is, the more merge operations are needed, as the develop-branch changes often on an hourly or daily basis. The developer has to stay up-to-date with the changes of the development branch at latest when the feature is ready.

The deployment to the test environment in the pipeline is made by merging the development branch to the release branch. When changes are pushed to the release branch, a build to the test environment is triggered automatically. After this, there may be some manual testing in this phase performed by for example the product owner or a user experience designer, or the different user roles at the customer. For some features, even some formal acceptance testing for the new features is needed before deploying them to the QA or production environments. In the context of this research, this latency in testing is considered to be waste, because when the feature is ready, a strict definition of done [16] would enforce it to be done in the sense of “ready for production”.

The deployment to the production environment happens by merging the release branch to the default branch and then building and deploying the project with a single click in the CI. When a manual step is required, it is easier for the team to keep control of the publishing schedule. Another manual step is to move the tickets on the Kanban board between the columns. In the case project, the schedule for production deployments is agreed on with the customer in weekly product owner team meetings, where involves both the supplier and the customer.

To summarize, the three wastes of the case project are:

- *Extra Processing Steps:* Testing procedure is too long.
- *Waiting:* Features are waiting for use in production environment, as users do not know about them.
- *Defects:* There are defects in the production environment not caught by developer tests.

Feature Flow: During the two month research period, the team completed a total number of 38 new features (Table I). The number of features that contained new API interfaces, and were thus fully trackable through the logs in the monitoring system, was 5. The mean development days per feature was 5 working days. Thus, on average, if a developer starts devel-

opment on a new feature today, the same developer can start the implementation of another feature on the same weekday next week.

Table I
METRICS DURING THE RESEARCH PERIOD.

	Development (days)	Deployment latency (days)	Business value latency (days)	Flow efficiency	Development done to production use (days)
f1	10	0	1	91	1
f2	4	0	1	80	1
f3	3	0	1	75	1
f4	1	2	1	25	3
f5	7	3	1	64	4
f6	7	7	1	47	8
f7	4	2	1	57	3
f8	1	4	1	17	5
f9	11	4	1	69	5
f10	1	5	1	14	6
f11	12	5	1	67	6
f12	1	5	1	14	6
f13	2	7	1	20	8
f14	2	8	1	18	9
f15	1	8	1	10	9
f16	9	9	1	47	10
f17	1	10	1	8	11
f18	7	10	1	39	11
f19	2	11	2	13	13
f20	7	12	1	35	13
f21	2	8	1	18	9
f22	9	8	2	47	10
f23	10	8	1	53	9
f24	2	10	1	15	11
f25	10	11	1	45	12
f26	1	14	1	6	15
f27	5	17	1	22	18
f28	10	17	1	36	18
f29	1	18	1	5	19
f30	2	18	1	10	19
f31	10	18	0	36	18
f32	6	19	1	23	20
f33	9	20	1	30	21
f34	1	20	1	5	21
f35	2	24	1	7	25
f36	2	1	1	50	2
f37	1	1	1	33	2
f38	2	1	1	50	2

The mean latency in deployment to production was 9 working days. The value is high, as any extra waiting prolongs the lead time in vain, which leads to the problems related to a long lead time described in Section III. The mean latency for production use for the five features that it was possible to collect the data for was 3 working days. Median was 1 working day and mode was 0 working days. We assume that latency of one working day is the best assumption of typical production use to occur, i.e., when the new feature is deployed, it will be used the next day. Assuming a production use latency value of 1 working day for the rest 33 features, the mean of flow efficiency was 34% and the mean time from completed implementation to production use was 10 working days. In other words, when the development was done, it took two weeks to get actual feedback from the production environment from the real users using the new features. Most of the latency occurs in the deployment phase (9 out of 10 days).

Summary: A modern development toolset includes a deployment pipeline built on top of DVCS. Such a toolset enables rapid and continuous development, while the obstacles of

the rapid development are visualized and eliminated, thus continuously optimizing the workflow. The demonstration with VSM and the feature flow efficiency shows that the flow efficiency can be improved simply by deploying more often. With frequent deployment, waste is eliminated. Lowering the latency of deployment leads to faster use and shorter feedback cycle.

VI. CONCLUSIONS

We studied the flow efficiency of a software project, with focus on the lead time from development done to production use. The goal was to recognize waste in a state-of-the-art deployment pipeline combined to a lean software development process. The results show that when the lead time is longer, the amount of waste increases in terms of context switching, starting unfinished tasks anew, and longer feedback cycles. In contrast, together with rapid deployment of new software, the ability to manage the lean flow efficiency results in shorter feedback cycles, fewer unfinished tasks and context switches, and, eventually, eliminating waste.

ACKNOWLEDGEMENTS

This work is supported by Tekes (<http://www.tekes.fi/>) and Digile's Need for Speed program (<http://www.n4s.fi/>). Thanks to Antti Virtanen for the idea of considering lead time to cover the time until the first production use.

REFERENCES

- [1] M. Poppendieck and T. Poppendieck, *Lean software development: an agile toolkit*. Addison-Wesley Professional, 2003.
- [2] K. B. Stone, "Four decades of lean: a systematic literature review," *International Journal of Lean Six Sigma*, vol. 3, no. 2, pp. 112–132, 2012.
- [3] M. Rother and J. Shook, *Learning to see: value stream mapping to add value and eliminate muda*. Lean Enterprise Institute, 2003.
- [4] J. Humble and D. Farley, *Continuous delivery: reliable software releases through build, test, and deployment automation*. Pearson Education, 2010.
- [5] J. Humble, C. Read, and D. North, "The deployment production line," in *Agile Conference*. IEEE, 2006, pp. 6–pp.
- [6] D. Feitelson, E. Frachtenberg, and K. Beck, "Development and deployment at facebook," *IEEE Internet Computing*, p. 1, 2013.
- [7] N. Modig and P. Åhlström, *This is lean: Resolving the efficiency paradox*. Rheologica, 2012.
- [8] R. C. Martin, *The clean coder: a code of conduct for professional programmers*. Pearson Education, 2011.
- [9] R. K. Yin, *Case study research: Design and methods*. Sage publications, 2014.
- [10] J. Boeg, *Priming Kanban: A 10 step guide to optimizing flow in your software delivery system*. Trifork, 2011.
- [11] P. Runeson and M. Höst, "Guidelines for conducting and reporting case study research in software engineering," *Empirical software engineering*, vol. 14, no. 2, pp. 131–164, 2009.
- [12] V. Driessen, "A succesful git branching model." <http://nvie.com/posts/a-successful-git-branching-model/>, retrieved: November 2014.
- [13] H. Kniberg, *Lean from the trenches: Managing large-scale projects with Kanban*. Pragmatic Bookshelf, 2011.
- [14] M. Pikkariainen and A. Mäntyniemi, "An approach for using cmmi in agile software development assessments: experiences from three case studies," in *SPICE 2006 conference, Luxembourg*, 2006, pp. 4–5.
- [15] C. Brindescu, M. Codoban, S. Shmarkatiuk, and D. Dig, "How do centralized and distributed version control systems impact software changes?" Corvallis, OR: Oregon State University, Dept. of Computer Science, Tech. Rep., 2014.
- [16] K. Schwaber and M. Beedle, "Agile software development with scrum. 2001," *Upper Saddle River, NJ*, 2003.